NEXORA
Autonomous AI Predictive Analytics Platform

Code Contribution Guide

Fork-and-Pull Workflow for Contributors

github.com/jeet2005/Nexora · MIT License

OVERVIEW

Nexora follows a standard **fork-and-pull** workflow for all code contributions. Contributors work on their own copy of the repository and submit changes back via a pull request. This ensures the main branch remains stable and all changes pass review before merging.

The complete workflow consists of eight steps:

1. Fork the repository on GitHub
2. Clone your fork locally
3. Create a feature branch from `main`
4. Make your modifications
5. Verify changes with tests, linting, and formatting
6. Commit your changes using Conventional Commits
7. Push the branch to your fork
8. Submit a Pull Request targeting the `main` branch of the upstream repository

---

1. FORK THE REPOSITORY

Navigate to the Nexora repository on GitHub and click **Fork** in the top-right corner. This creates a personal copy of the repository under your GitHub account. All development work happens in your fork — the upstream repository remains unchanged until a pull request is merged.

2. CLONE YOUR FORK LOCALLY

Clone your fork to your local machine, then add the upstream remote so you can pull future changes from the main project.

```
# Clone your fork (replace YOUR_USERNAME with your GitHub handle)
$ git clone https://github.com/YOUR_USERNAME/Nexora.git
$ cd Nexora

# Add the upstream remote
$ git remote add upstream https://github.com/jeet2005/Nexora.git

# Verify remotes
$ git remote -v
# origin    https://github.com/YOUR_USERNAME/Nexora.git  (fetch)
# origin    https://github.com/YOUR_USERNAME/Nexora.git  (push)
# upstream  https://github.com/jeet2005/Nexora.git       (fetch)
# upstream  https://github.com/jeet2005/Nexora.git       (push)
```

> **Keeping your fork in sync.** Before starting any new work, pull the latest changes from upstream into your local `main` branch to avoid conflicts.

```
# Sync your fork with upstream before new work
$ git checkout main
$ git fetch upstream
$ git merge upstream/main
$ git push origin main
```

3. CREATE A FEATURE BRANCH

All changes must be made on a dedicated branch created from `main`. Never commit directly to `main`. Branches should be short-lived and focused on a single concern — one feature or one fix per branch.

```
# Always branch from an up-to-date main
$ git checkout main
$ git pull upstream main

# Create your branch
$ git checkout -b feature/your-feature-name
```

Branch Naming Conventions

Prefix	When to use	Example
`feature/`	New functionality, UI additions, new endpoints	`feature/add-csv-export`
`fix/`	Bug fixes, regressions, crash patches	`fix/auth-token-expiry`
`docs/`	Documentation only — README, guides, comments	`docs/update-setup-guide`
`refactor/`	Code restructuring with no behavior change	`refactor/model-service-layer`
`chore/`	Config changes, CI tweaks, dependency bumps	`chore/upgrade-fastapi-0.110`

> Branch names use lowercase kebab-case only. No spaces, no uppercase, no special characters except hyphens. Keep them under 40 characters.

4. MAKE YOUR MODIFICATIONS

Implement your changes within the scope of the branch. Keep changes focused — avoid mixing unrelated fixes or features into the same branch. If you discover an unrelated bug while working, open a separate issue or branch for it.

5. VERIFY CHANGES

Before committing, run the full quality suite: tests, linting, and formatting. All checks must pass locally before pushing. GitHub Actions CI will run the same checks on your pull request and will block merging if any fail.

Backend (Python / FastAPI)

```
# From /backend
$ pytest tests/ -v          # run tests
$ ruff check .              # lint
$ ruff format .             # format (auto-fixes in place)
$ mypy app/                 # optional: type check
```

Frontend (React / Vite)

```
# From /frontend
$ npx vitest run            # run tests
$ npx eslint src/           # lint
$ npx prettier --write src/ # format
$ npm run build             # catch TypeScript errors
```

Pre-commit Checklist

- [ ] All tests pass locally with no failures
- [ ] No linting errors (`ruff check .` / `eslint src/` return zero issues)
- [ ] Code is formatted (`ruff format .` / `prettier --write .` ran)
- [ ] New logic has corresponding unit or integration tests
- [ ] No debug statements (`print()` / `console.log()`) left in code
- [ ] No hardcoded secrets, API keys, or local file paths
- [ ] Existing tests still pass — no regressions introduced

6. COMMIT USING CONVENTIONAL COMMITS

Nexora follows the **Conventional Commits** specification. Every commit message must use the format: `type(scope): description`. The type describes the kind of change; the scope is optional but recommended. The description should be present-tense, lowercase, and under 72 characters.

Type	When to use
`feat`	A new feature or user-visible addition
`fix`	A bug fix
`docs`	Documentation changes only
`test`	Adding or updating tests
`refactor`	Code restructuring with no behavior change
`chore`	Build, config, or dependency changes
`style`	Formatting only, no logic change

```
# Examples of valid commit messages
feat(api): add CSV export endpoint for predictions
fix(dashboard): resolve null pointer on empty model list
docs: update local setup instructions in README
test(auth): add unit tests for JWT refresh flow
refactor: extract model loading into dedicated service
chore: upgrade fastapi to 0.110.0

# For breaking changes — add ! and a footer
feat(api)!: rename /predict to /inference
# BREAKING CHANGE: /predict endpoint has been renamed to /inference
```

7. PUSH THE BRANCH TO YOUR FORK

```
# Stage and commit
$ git add .
$ git commit -m "feat(dashboard): add real-time prediction confidence chart"

# Push to your fork
$ git push origin feature/your-feature-name
```

8. SUBMIT A PULL REQUEST

After pushing your branch, GitHub will display a prompt to open a pull request. The PR must target the `main` branch of `jeet2005/Nexora` — not your fork's main. Never target a release or development branch unless a maintainer specifically requests it.

Steps to Open a PR

1. Go to your fork on GitHub after pushing. Click the **Compare & pull request** banner, or navigate to Pull Requests and click **New Pull Request**.
2. Set the base repository to `jeet2005/Nexora` and the base branch to `main`.
3. Write a clear description: what changed, why it changed, and how to test it. Link the related issue with `Closes #123` if applicable.
4. Attach screenshots for any visual or UI changes.
5. Submit and wait for CI to complete. A maintainer will review and may request changes. Push fixes to the same branch — the PR updates automatically.

PR Description Template

```
## What does this PR do?
Brief summary of the change and the motivation behind it.

## Related issue
Closes #ISSUE_NUMBER  (remove line if no related issue)

## How to test
1. Run docker-compose up
2. Navigate to /dashboard
3. Verify the export button appears and downloads correctly

## Screenshots (if UI changes)
Before: [screenshot]
After:  [screenshot]

## Checklist
- [ ] Tests pass locally
- [ ] No linting errors
- [ ] Code is formatted
- [ ] PR targets the main branch of jeet2005/Nexora
```

> **First-time contributors:** A welcome message will appear automatically on your first PR. If you are unsure about anything, open a Draft PR early and ask questions in the description. Maintainers are here to help.